1 Introduction

During this phase of the project, the team concentrated on developing code to be executed directly from the car and designing computer vision algorithms essential for extracting visual cues.

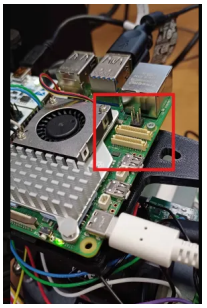
2 Planned activities

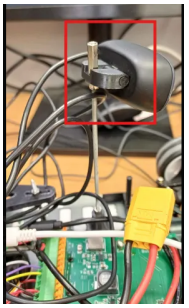
- Move the vehicle by code in the raspberry pi
- Get the raspberry camera running
- Develop a simple lane detection algorithm
- Calculate and follow a makeshift trajectory based on the detected lanes.

3 Status of planned activities

➔ **Move the vehicle by code in the raspberry pi.**
Status: Completed
Implementation: Created new component using the provided script, used the MessageHandlerSubscriber and MessageHandlerSender classes to establish communication with the four queues handled by the Gateway process and finally sent commands after making sure that the engine has been activated.

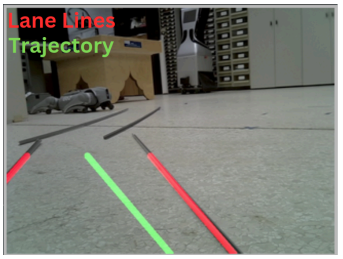
➔ **Get the raspberry camera running.**
Status: Canceled
Implementation: The raspberry's camera was unusable as the dedicated ports on the raspberry pi were damaged, so we used a Logitech C505e instead and connected it to the Dashboard as well.





➔ **Develop a simple lane detection algorithm.**
Status: Completed
Implementation: Applying an edge detection filter (Canny) to the camera's feed and the Hough transform, we are able to detect the lines present in the image. Afterwards, using a "mask" that excludes all values that appear far from the vehicle's front and tweaking the thresholding values of the filters, the weak Hough responses are eliminated, leaving only the two strongest responses as the detected "lanes".

➔ **Calculate and follow makeshift trajectory based on the detected lanes.**
Status: Completed
Implementation: The trajectory is calculated as a line connecting the current front of the vehicle, as seen from the camera, and the point located in the top middle of the lane lines. The car tries to follow this line, by moving constantly forward and adjusting its steering value according to the angle of the calculated trajectory line.



Complications:

- The Logitech camera used has limited Field of View, meaning that the lane lines even when in front of the car are sometimes not visible. Additionally, the mount for the camera we 3D printed is unstable and causes the camera to shift during the car's movement.

- Due to a bad network, there is outrageous latency on the camera's feedback displayed on the dashboard. Which makes testing time consuming.

- The lane line detection algorithm is susceptible to outliers messing up the calculations.
  Objects in the environment that contain lines and are picked up by the camera and get through the "mask" applied, are creating impossible trajectories. A more sophisticated algorithm must be used, along with a more capable camera.

4 General status of the project

The vehicle is able to follow a straight line, and correct its course accordingly. However this is only achieved in ideal testing conditions (minimal outliers) and extremely narrow lanes (because of the camera's limited Field of View).

5 Upcoming activities

- Craft a proper track for testing.
- Replace the camera with a better one and adapt the algorithm.
- Implement odometry to keep track of the vehicle's position.
- Apply some filter (i.e. Kalman) to ensure robustness to outliers in the trajectory's calculation.
- Introduce PID control to avoid sudden movements.